

# Data Structures Midterm Exam

(Example Exam)

Name: \_\_\_\_\_

*Read each question carefully and fully before answering. Answer all questions using complete sentences unless the question specifies otherwise. Explain your thoughts and provide any necessary context; supporting examples, code, formulas, etc. are encouraged. Clearly state any additional assumptions you make in forming your responses; always be as specific as possible.*

## Part 1: Multiple Choice (24 pts)

Choose the best answer for each question. (4 pts each)

*Write any assumptions you make in the margin.*

1. What is the worst-case time complexity of searching in a balanced binary search tree?
  - a.  $O(1)$
  - b.  $O(\log n)$
  - c.  $O(n)$
  - d.  $O(n \log n)$
  
2. Which data structure would be most efficient for implementing a browser's back button?
  - a. Queue
  - b. Stack
  - c. Binary tree
  - d. Hash table
  
3. What is the primary advantage of a linked list over an array?
  - a. Faster random access
  - b. Better cache locality
  - c. Dynamic size without reallocation
  - d. Less memory overhead
  
4. In a max heap, where is the largest element always located?
  - a. At the root
  - b. At a leaf node
  - c. In the middle level
  - d. At any position
  
5. What is the average-case time complexity of hash table lookups with a good hash function?
  - a.  $O(1)$
  - b.  $O(\log n)$
  - c.  $O(n)$
  - d.  $O(n^2)$

6. Which traversal of a binary search tree visits nodes in sorted order?
- a. Pre-order
  - b. In-order
  - c. Post-order
  - d. Level-order

## Part 2: Fill in the Blank (16 pts)

Complete each statement with the appropriate term. (2 pts each)

1. A \_\_\_\_\_ is a linear data structure that follows FIFO (First In, First Out) ordering. \_\_\_\_
2. The operation that adds an element to a stack is called \_\_\_\_, while removing an element is called \_\_\_\_.
3. In a binary tree, a node with no children is called a \_\_\_\_ node.
4. The process of visiting all nodes in a tree is called \_\_\_\_.
5. A \_\_\_\_ is a special tree-based data structure that satisfies the heap property.
6. The \_\_\_\_ of a tree is the length of the longest path from root to a leaf.

### **Part 3: Short Answer** (30 pts)

Answer each question in the space provided. Write clearly and concisely. (10 pts each)

1. (10 pts) Explain the difference between a stack and a queue. Give one real-world application for each.

Now continue to the following questions.

2. (10 pts) What is a hash collision?  
Describe one method for handling hash collisions.

3. (10 pts) Explain why binary search requires a sorted array. What happens if the array is not sorted?

## Part 4: Algorithm Analysis (20 pts)

### Reference: Big O Complexity Classes

- $O(1)$  - Constant time
- $O(\log n)$  - Logarithmic time
- $O(n)$  - Linear time
- $O(n \log n)$  - Linearithmic time
- $O(n^2)$  - Quadratic time

Consider the following Python function:

```
def find_duplicates(arr):
    duplicates = []
    for i in range(len(arr)):
        for j in range(i + 1, len(arr)):
            if arr[i] == arr[j] and arr[i] not in duplicates:
                duplicates.append(arr[i])
    return duplicates
```

1. (8 pts) What is the time complexity of this function? Explain your reasoning.
2. (12 pts) Describe how you could improve the time complexity using a different data structure. What would the new time complexity be?

## Part 5: Implementation (10 pts)

### Example: Reversing a String

```
def reverse_string(s):  
    return s[::-1]  
  
# Usage:  
print(reverse_string("hello")) # Output: "olleh"
```

Write a function `is_palindrome(s)` that checks if a string is a palindrome (reads the same forwards and backwards). Ignore spaces and capitalization.

```
def is_palindrome(s):  
    # Your implementation  
    here  
    pass  
  
# Test cases:  
# is_palindrome("racecar") → True  
# is_palindrome("hello") → False  
# is_palindrome("A man a plan a canal Panama") → True
```

### Sample Solution:

```
def is_palindrome(s):  
    # Remove spaces and  
    convert to lowercase  
    cleaned = s.replace(" ",  
        "").lower()  
    # Compare with reversed  
    version  
    return cleaned ==  
        cleaned[::-1]
```

**Explanation:** We first clean the string by removing spaces and converting to lowercase, then compare it with its reversed version using Python's slice notation `[::-1]`.